TITAI v12

Phenomenal Improvement, Production Convergence, and Breakthrough Engineering Report

Evidence-backed engineering release

DEVELOPMENT ONLY

Prepared 11 July 2026
English-only canonical package

1. Executive Summary

Release decision: DEVELOPMENT ONLY

v12 produces a measurable step change in execution safety and operator truth. It adds stale-worker fencing, guarded step-level recovery, approval compare-and-set concurrency, verification-aware routing, governed memory hardening, risk-adjusted autonomy, immutable verification attestations, a conservative pilot evidence gate, dynamic Roast, property/race testing, mutation probes, and an Evidence Control Plane design thesis.

It does not claim controlled-pilot or production readiness because native persistence tests, full build completion, browser E2E and accessibility, live-provider restart continuation, complete tenant/workspace isolation, and independent verification are not proven.

Evidence gate	Result	Interpretation
Canonical integrity	**PASS**	65 tasks / 68 claims / 26 ADRs
Safe regression	**PASS**	40 files / 140 tests
Baseline evaluation	**PASS**	8 of 8
Crash recovery proof	**PASS**	2 attempts / exactly 1 side effect
Static Roast	**PARTIAL**	43 of 44; Drizzle remains open
Dynamic Roast	**PASS**	6 of 6 modes
Mutation probes	**PASS**	6 of 6 curated mutants killed
Native SQLite suites	**BLOCKED**	Binding absent; no tests collected
Production build	**BLOCKED**	Compiled/typechecked; page-data stage timed out
Browser/accessibility	**BLOCKED**	No real browser conformance evidence

2. Report Map

- Verified starting condition
- Architecture changes
- Security, tenancy, and authority
- Durable execution and recovery
- Governed memory and model routing
- Tool security and verification
- Breakthrough engineering lab
- Design tournament and selected thesis
- Accessibility and cognitive reliability
- Performance, cost, and energy evidence
- Unknown unknowns
- Complete Roast finding register
- Supply chain and release evidence
- Honest limitations and next work

3. Verified Starting Condition

Input ZIP SHA-256: a7352c880b455c1a71a389d32e27e997bc9e61bddd93319f6b564adb5c4097e2

- Next.js/TypeScript application with SQLite/Drizzle persistence and API routes.
- Agent/provider abstractions, policy-gated tools, approvals, governed memory, event and audit streams.
- Portable durable core, evidence-oriented run dossier, operations signal plane, backup and restore.

- All safe baseline gates were run before v12 implementation.

4. Architecture Changes

Step-level durable execution

Revisioned durable workflows now model dependencies, attempts, fencing-token leases, checkpoints, receipts, reconciliation, and terminal finalization. A stale worker cannot commit after lease ownership changes.

Guarded active-runner cutover

Simulation and dry-run workflows can use step-level recovery. Live-provider execution remains guarded because provider conversation continuation and arbitrary external side effects are not serialized end to end.

Approval concurrency

Approval rows now carry a revision. API decisions require expectedRevision and use compare-and-set updates; stale clients receive an explicit conflict.

Verification domain

Verification attestations bind task and requirement revisions, source and artifact hashes, builder and verifier identities, limitations, and release candidate identity.

Model and memory controls

Model routing includes health and verification quality. Governed memory applies normalization, quarantine, ownership, sensitivity, expiry, authority, contradiction, and provenance controls.

Risk-adjusted authority

Tool autonomy is derived from authenticated identity, scope, capability, policy, approval, side-effect class, idempotency, and recovery semantics - never from model confidence alone.

Operator truth surfaces

Run completion, verification, and pilot readiness are represented as separate claims in the Evidence Control Plane.

5. Security, Identity, Tenancy, and Authority

- Centralized resource-access and verifier-separation primitives were added.
- Approval races are mitigated with persisted revisions and compare-and-set transitions.
- Unavailable model candidates fail hard constraints; degraded health is explicit.
- Memory can inform decisions but cannot grant capabilities, approve effects, change scope, or verify work.
- Tool contracts include risk, effect, timeout, output ceiling, idempotency, network/filesystem access, and approval requirements.
- Full tenant/workspace semantics and strong process isolation remain open.

6. Durable Execution and Recovery

Crash proof evidence hash: 1f804c465c4e6e5ba3dfaae79b25298150c73c37459efc2022b49ed9c7d4613e

- Injected crash was observed.
- A second worker acquired a later fencing token.
- The stale worker was unable to commit.

- Exactly one side effect occurred across two attempts.
- Checkpoint and receipt state allowed deterministic recovery.

Boundary: The proof covers the portable/guarded path. It is not evidence of arbitrary live-provider conversation recovery.

7. Governed Memory and Model Routing

- Memory text is Unicode-normalized, stripped of zero-width ambiguity, inspected for instruction-like content, and provenance-labeled.
- Contradictory records are ranked by authority, confidence, freshness, risk, and task relevance.
- Ablation scenarios cover no memory, raw semantic memory, governed memory, poisoning, contradictions, sensitivity, and token pressure.
- Routing applies hard eligibility constraints before utility scoring, including health and verification quality.
- Routing is deterministic under candidate-order permutations and can return no eligible model.

8. Verification That Cannot Be Implied

- Verification records are distinct from run completion and implementation status.
- Attestations bind evidence and source hashes, builder and verifier identities, task revision, limitations, and release candidate.
- Obvious builder/verifier identity collision is rejected.
- No separate verifier operated in this execution, so no final artifact is labeled independently verified.

9. Breakthrough Engineering Lab

BH-001 - Verification-aware model routing

Mechanism: Weight measured verifier quality and model health for review tasks.

Benefit: Reduce weak-review selection.

Failure mode: Self-reported scores may be inaccurate.

Experiment: Compare verifier-task selection under competing builder and verifier profiles.

Success / kill: Eligible high-verification model wins deterministically. / Routing remains unchanged or selects unavailable model.

Status: implemented_tested

BH-002 - Task risk-adjusted autonomy

Mechanism: Derive autonomy from risk, effect, idempotency, reversibility, evidence, and human authority.

Benefit: Increase safe autonomy for reads while constraining dangerous effects.

Failure mode: Scoring can create false precision.

Experiment: Challenge dry-run writes, external effects, low-risk reads, and prompt-derived authority.

Success / kill: Unsafe cases deny or require human control; bounded reads remain autonomous. / Any prompt-derived authority or dry-run side effect is permitted.

Status: implemented_tested

BH-003 - Evidence-weighted agent planning

Mechanism: Prefer plans whose steps declare evidence and verification outputs.

Benefit: Reduce untestable work.

Failure mode: Agents may game evidence declarations.

Experiment: Compare completion correctness on equal-length plans with and without evidence requirements.

Success / kill: Higher verified-task rate without excessive latency. / More ceremony without higher verification rate.

Status: designed

BH-004 - Counterfactual recovery simulation

Mechanism: Simulate failure at each checkpoint before approving a high-risk workflow.

Benefit: Expose unrecoverable paths before execution.

Failure mode: Simulation model may omit real side effects.

Experiment: Compare predicted recovery action with injected-failure outcomes.

Success / kill: No unsafe-retry mismatch across the selected matrix. / Predictions diverge materially from injected outcomes.

Status: designed

BH-005 - Memory confidence decay

Mechanism: Reduce authority as evidence ages or conflicts accumulate.

Benefit: Lower stale-memory influence.

Failure mode: Valuable stable facts may decay unnecessarily.

Experiment: Evaluate stale and corrected memories over simulated time.

Success / kill: Lower contradiction rate without unacceptable recall loss. / Recall loss exceeds safety benefit.

Status: proposed

BH-006 - Reconciliation-first tool protocol

Mechanism: Persist intent before ambiguous external effects and require resolver-specific reconciliation.

Benefit: Avoid blind retries after uncertain outcomes.

Failure mode: Many tools cannot query effect status.

Experiment: Inject timeout after remote acceptance and compare retry behavior.

Success / kill: No duplicate effect and explicit operator action when status is unknowable. / Protocol still retries blindly.

Status: partially_implemented

BH-007 - Adaptive context knapsack

Mechanism: Optimize context under token budget while reserving mandatory policy and evidence slots.

Benefit: Lower cost and contamination.

Failure mode: Utility estimates may be wrong.

Experiment: Ablate fixed, semantic, and governed adaptive context.

Success / kill: Equal or better correctness with lower token cost and unsafe selection. / Correctness falls or policy context is displaced.

Status: partially_implemented

BH-008 - Signed verifier worker

Mechanism: Execute tests in a clean checkout and sign hashes with an independent service identity.

Benefit: Make verification hard to fake.

Failure mode: Signing infrastructure becomes a privileged bottleneck.

Experiment: Attempt stale-source and replaced-artifact verification.

Success / kill: Mismatched hashes are rejected and signatures verify offline. / Service can attest mutable or unexecuted evidence.

Status: proposed

BH-009 - Energy-aware local-first routing

Mechanism: Include measured energy and idle cost in the routing Pareto frontier.

Benefit: Improve sovereign efficiency.

Failure mode: Energy estimates may be too coarse.

Experiment: Measure fixed local workloads on representative hardware.

Success / kill: Lower energy per verified task without quality regression. / Measurement noise dominates decisions.

Status: designed

BH-010 - Operator uncertainty visualization

Mechanism: Represent evidence gaps, unknowns, and reconciliation as first-class visual states.

Benefit: Reduce false confidence and unsafe retries.

Failure mode: Additional signals may increase cognitive load.

Experiment: Timed walkthrough comparing status-only and evidence-control interfaces.

Success / kill: Faster correct identification of next safe action. / No comprehension gain or higher operator error.

Status: implemented_partial

10. Product and Design Transformation

Selected thesis: Evidence Control Plane with Recovery-First Operations and a Human-Authority boundary. Completion, verification, release readiness, and runtime health remain visually and semantically separate.

Nine structurally different directions were evaluated. Scores are disciplined design estimates, not user-study evidence.

Direction	Mental model	Score	Decision
Evidence Control Plane	A run is a chain of claims, authorities, and evidence.	86/100	selected_core
Operational Command Map	A live map of jobs, workers, queues, and recovery debt.	77/100	selected_operations_layer
Temporal Execution Ledger	A chronological, append-only operational record.	79/100	retained_component
Dependency Graph Workspace	Tasks and agents are a navigable DAG.	60/100	rejected_as_primary
Verification Dossier	Each run is an auditable case file.	84/100	selected_verification_layer
Multi-Pane Engineering Cockpit	Persistent simultaneous views of state, evidence, and action.	71/100	rejected_as_primary
Canonical Document Workspace	Decisions and tasks are executable documents.	76/100	retained_for_governance
Recovery-First Operations Console	Every state is organized around safe recovery actions.	84/100	selected_recovery_layer
Human-Authority Control Surface	The primary object is the boundary between machine proposal and human authority.	84/100	selected_approval_layer

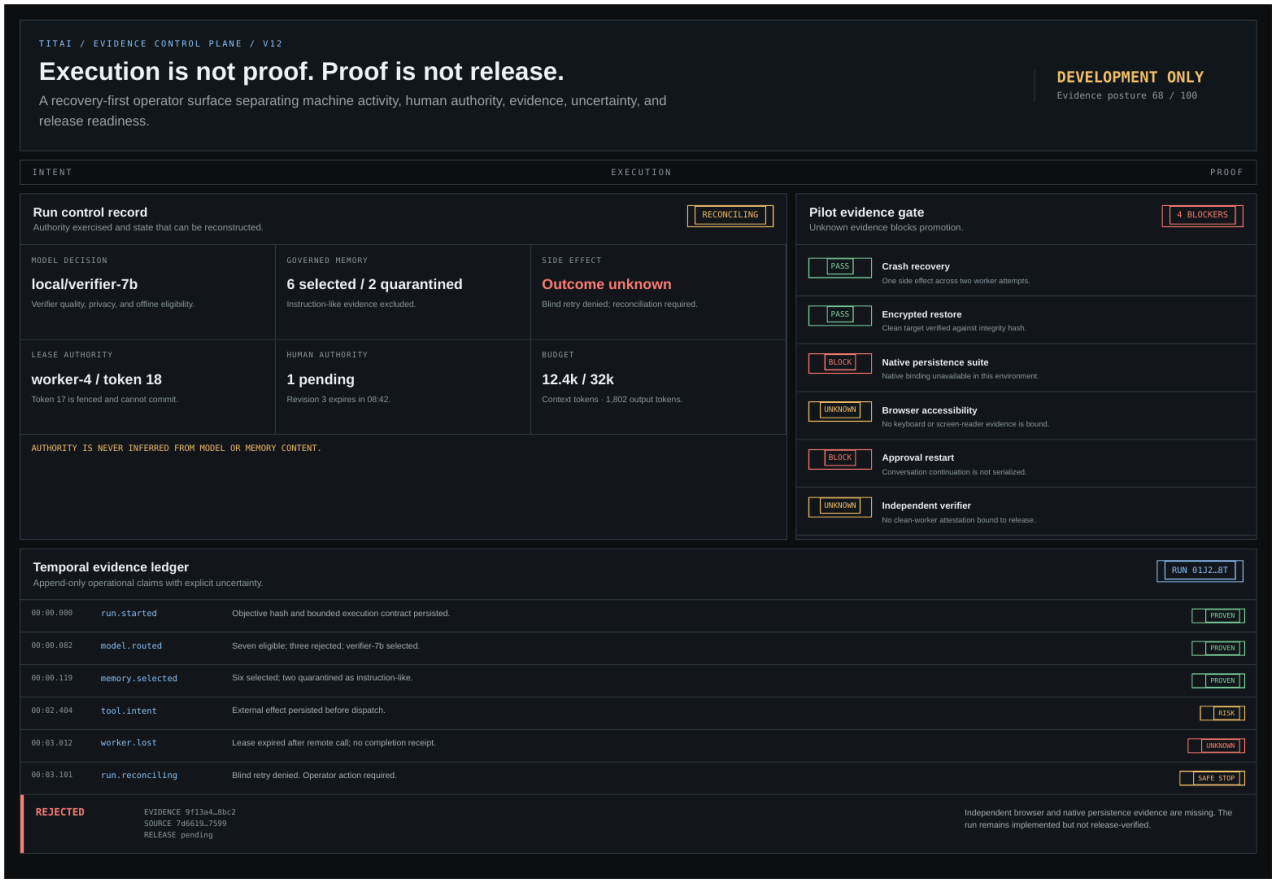


Figure 1. Deterministic visual evidence for the Evidence Control Plane. This is not browser E2E evidence.

11. Accessibility and Cognitive Reliability

- Source-contract tests require named regions, non-color status words, reduced motion, narrow-width rules, and semantic list structure.
- Verification, failure, recovery, reconciliation, and unknown evidence are visible text states rather than color-only decoration.
- Browser keyboard, screen-reader, zoom, focus restoration, and visual-regression evidence are not available in this environment.

12. Performance, Capacity, Cost, and Energy Evidence

Deterministic local microbenchmarks. They are not production load tests and do not measure model or network latency.

Benchmark	p50 ms	p95 ms	p99 ms	max ms	mean ms
validate_30_step_dag	0.0666	0.3248	0.7652	3.1162	0.1131
route_24_model_candidates	0.0331	0.0862	0.1681	0.9617	0.045
govern_200_memory_candidates	1.5478	3.1303	3.3759	9.5746	1.8292

- No production load, cloud cost, or real hardware energy claim is made.

- The bounded build generated excessive page-data workers and was terminated; this remains an operational finding.

13. Unknown Unknowns Ledger

UU-001 - CRITICAL - verified_gap

Hypothesis: The active durable runner is step-recoverable only for simulation or dry-run execution.

Experiment: Inspect the startRun durable cutover guard and run-mode tests.

Result: Non-simulation, non-dry-run durable execution is rejected.

Next action: Prove live-provider tool continuation and side-effect reconciliation before removing the guard.

UU-002 - HIGH - verified_gap

Hypothesis: Durable recovery scheduling depends on an HTTP list request rather than a dedicated worker loop.

Experiment: Trace all recoverDurableRuns call sites.

Result: Recovery is opportunistically triggered from the runs API.

Next action: Add a bounded worker entrypoint with heartbeat, shutdown, backpressure, and deployment ownership.

UU-003 - CRITICAL - verified_gap

Hypothesis: Approval state is durable but the active model conversation is not restart-continuable.

Experiment: Resolve an approval after the active process entry is absent.

Result: The run is intentionally marked interrupted because conversation continuation is not serialized.

Next action: Persist conversation, tool intent, iteration, and step checkpoint or redesign execution around message-driven turns.

UU-004 - HIGH - verified_gap

Hypothesis: Model capability values can be self-entered without signed benchmark provenance.

Experiment: Inspect the model-profile API and routing evidence model.

Result: Profiles accept bounded scores but do not require benchmark artifact hashes or signed evaluators.

Next action: Bind promotion-eligible scores to immutable evaluation runs and evaluator identity.

UU-005 - HIGH - verified_gap

Hypothesis: Verification attestations prove integrity of supplied evidence references but do not execute or independently acquire the tests.

Experiment: Inspect attestation creation and API behavior.

Result: The verifier supplies hashes and a command; integrity is checked, execution provenance is not automatically acquired.

Next action: Create a verifier worker that checks out the source hash, executes the command, and signs the result.

UU-006 - HIGH - verified_gap

Hypothesis: Owner scoping is not a complete tenant and workspace authorization model.

Experiment: Compare the authorization contract with persisted resource columns.

Result: Most resources have owner scope; tenant and workspace identity are not persisted end-to-end.

Next action: Define deployment mode explicitly or add tenant/workspace columns, centralized policy, migrations, and cross-tenant tests.

UU-007 - MEDIUM - verified_gap

Hypothesis: The CSP still permits inline scripts and styles.

Experiment: Inspect the generated Content-Security-Policy.

Result: Both script-src and style-src include unsafe-inline for framework compatibility.

Next action: Evaluate nonce or hash-based CSP during a real production build and browser E2E run.

UU-008 - HIGH - verified_gap

Hypothesis: The tool broker is policy-rich but tools still execute inside the application process.

Experiment: Trace executeBoundedTool and tool implementations.

Result: Timeout and output bounds exist, but there is no OS-level isolation boundary.

Next action: Introduce replaceable WASI, container, or microVM executors for high-risk tools.

UU-009 - HIGH - environment_blocked

Hypothesis: SQLite transactional behavior and API integration remain unproven in this package without the native binding.

Experiment: Collect native-dependent test suites.

Result: The suites cannot load better_sqlite3.node after lifecycle scripts were intentionally disabled.

Next action: Run preserved suites on an approved development or CI machine without weakening endpoint protection.

UU-010 - MEDIUM - verified_gap

Hypothesis: Feature flags and compatibility modes may remain indefinitely and diverge.

Experiment: Inspect runner mode definitions and canonical tasks for removal criteria.

Result: Modes are explicit but do not yet have measured promotion and retirement dates.

Next action: Attach shadow-divergence thresholds and removal criteria to the durable cutover task.

UU-011 - MEDIUM - verified_gap

Hypothesis: Local metrics cannot prove external availability or production load behavior.

Experiment: Inspect the operations metrics data sources.

Result: Metrics are derived from local persisted observations without external probes or distributed traces.

Next action: Add privacy-controlled external probes or OpenTelemetry-compatible export as an optional adapter.

UU-012 - HIGH - verified_gap

Hypothesis: Audit integrity is local and not externally anchored.

Experiment: Inspect audit append and release provenance.

Result: The chain is transactionally appended but has no external transparency-log or signed checkpoint.

Next action: Add optional signed audit checkpoints and offline-verifiable export.

14. Roast 3.0 Summary

Total findings: 55

Severity: {'critical': 7, 'high': 37, 'medium': 11}

Status: {'partially_mitigated': 25, 'fixed_in_v8': 13, 'fixed_in_v10': 6, 'open': 1, 'environment_blocked': 2, 'fixed_in_v11': 3, 'fixed_in_v12': 5}

- Static Roast: 43/44 controls passed; the open control is the Drizzle security-baseline upgrade.

- Dynamic Roast: 6/6 runtime, recovery, security, product, release, and performance modes passed.
- Mutation: 6/6 curated semantic mutants killed; this is not an exhaustive AST mutation score.

15. Complete Roast Finding Register

F-001 - Detached in-memory execution cannot survive process loss

Severity	Status	Confidence	Category
critical	partially_mitigated	high	durability

Finding: A feature-flagged durable envelope and SQLite durable primitives now exist, but granular step resume and active restart recovery remain incomplete.

Impact: Long-running or side-effecting jobs can disappear, duplicate, or remain permanently inconsistent after deployment, crash, or upgrade.

Evidence: src/lib/engine/start.ts, src/lib/engine/runner.ts, src/lib/db/schema.ts

Recommendation: Complete native multi-process crash tests and replace the single-step envelope with granular checkpoints and fencing.

Acceptance test: Kill the worker at every checkpoint boundary and prove the run resumes exactly once with preserved ownership and evidence.

F-002 - Side effects lack persisted idempotency and atomic evidence

Severity	Status	Confidence	Category
critical	partially_mitigated	high	durability

Finding: The v9 portable core now persists deterministic side-effect receipts and proves crash replay without duplication. The production runner still lacks the SQLite transactional receipt, event, and audit integration.

Impact: A crash after a side effect but before acknowledgement can repeat irreversible work or create evidence that disagrees with reality.

Evidence: src/lib/engine/runner.ts, src/lib/engine/events.ts, src/lib/engine/audit.ts, src/lib/db/schema.ts, src/lib/durable/engine.ts, docs/canonical/evidence/durable-vertical-slice.json

Recommendation: Migrate the production runner to the portable contract with a SQLite compare-and-set lease and transactional outbox.

Acceptance test: Inject a crash after external success but before event commit and prove the retry does not repeat the effect.

F-003 - Deployment model conflicts with globally shared mutable data

Severity	Status	Confidence	Category
critical	partially_mitigated	high	identity-tenancy

Finding: Authentication is multi-user, but policies, templates, and parts of audit/configuration are global rather than tenant-owned. Administrative restrictions reduce some exposure but do not establish a complete tenancy model.

Impact: One user can be affected by another user's global configuration or data; future multi-user deployment risks cross-tenant leakage and confused authority.

Evidence: src/lib/db/schema.ts, src/app/api/policies/route.ts, src/app/api/templates/route.ts, src/app/api/audit/verify/route.ts

Recommendation: Choose and enforce either explicit single-owner mode or workspace/tenant ownership across every mutable record and query.

Acceptance test: Create two tenants and prove every read, write, audit, template, skill, run, and artifact is isolated under adversarial identifiers.

F-004 - HTTP and provider endpoints allowed server-side request forgery

Severity	Status	Confidence	Category

critical	fixed_in_v8	high	security-egress

Finding: User-influenced HTTP and provider base URLs previously had no private-network, loopback, metadata, DNS-rebinding, or redirect boundary.

Impact: An agent could probe internal services, cloud metadata, or local control planes.

Evidence: src/lib/security/egress.ts, src/lib/engine/tools/httpFetch.ts, src/lib/engine/gateway.ts, tests/egress.test.ts

Recommendation: Keep destination and redirect validation, DNS resolution checks, and explicit local-model exceptions under policy.

Acceptance test: Run the egress suite against loopback, RFC1918, link-local, metadata, IPv6, encoded addresses, mixed DNS answers, and redirect chains.

F-005 - Invalid or cyclic plans could be reported as completed

Severity	Status	Confidence	Category
critical	fixed_in_v8	high	runtime-correctness

Finding: The original scheduler could reach no-ready-step state while unfinished nodes remained and still finish the run successfully.

Impact: A structurally impossible plan could produce a false success signal and misleading evidence.

Evidence: src/lib/engine/plan.ts, src/lib/engine/runner.ts, tests/plan.test.ts

Recommendation: Retain preflight plan validation and fail closed when unfinished work has no executable node.

Acceptance test: Test duplicate nodes, unknown edges, self-dependencies, cycles, excessive steps, and runtime deadlock.

F-006 - No tested sovereign backup and restore path

Severity	Status	Confidence	Category
critical	fixed_in_v10	high	recovery

Finding: The v10 package creates a consistent SQLite snapshot, encrypts it with scrypt and AES-256-GCM, verifies integrity and required tables, restores atomically, preserves an overwritten database, and includes positive and negative restore tests.

Impact: Loss or corruption of the SQLite database, keys, artifacts, or model configuration can be unrecoverable.

Evidence: docs/canonical/20_DATA_LIFECYCLE_CONTRACT.md, docs/canonical/24_HONEST_LIMITATIONS.md, src/lib/durable/backup.ts, docs/canonical/evidence/durable-vertical-slice.json, src/lib/backup/application-backup.ts, scripts/backup-application.mjs, scripts/restore-application.mjs, tests/application-backup.test.ts, docs/canonical/decisions/ADR-0023-encrypted-application-database-backup.md

Recommendation: Run periodic clean-deployment RTO/RPO drills and store the passphrase separately from the backup.

Acceptance test: Restore the encrypted backup into a clean target, verify the sentinel data and database checks, then prove wrong-key, tamper, and unauthorized-overwrite attempts fail closed.

F-007 - Drizzle ORM remains on a version with a high-severity advisory

Severity	Status	Confidence	Category
high	open	high	supply-chain

Finding: The current dependency range resolves below the audited fixed line identified by npm audit for identifier-related SQL injection.

Impact: Unsafe identifier construction in affected paths can turn application input into SQL structure rather than data.

Evidence: package.json, docs/roast/evidence/npm-audit.json

Recommendation: Upgrade Drizzle ORM to a supported non-vulnerable release, regenerate/migrate safely, and execute database integration tests.

Acceptance test: npm audit contains no high advisory for Drizzle and all migration/query/integration tests pass on a clean install.

F-008 - Any authenticated user could mutate global policy

Severity	Status	Confidence	Category
high	fixed_in_v8	high	authorization

Finding: The policy update path previously had an ineffective administrative check while changing system-wide policy.

Impact: A normal user could weaken approval requirements for all users.

Evidence: src/app/api/policies/route.ts

Recommendation: Retain explicit administrative authorization and later tenant-scope policy records.

Acceptance test: A non-admin receives a denial and cannot change persisted policy; an authorized admin creates an attributable audit event.

F-009 - Skill identifiers enabled cross-user prompt access or attachment

Severity	Status	Confidence	Category
high	fixed_in_v8	high	authorization

Finding: Direct skill fetch and agent skill replacement did not consistently verify ownership.

Impact: A user who guessed an identifier could read or attach another user's procedural prompt material.

Evidence: src/app/api/skills/[skillId]/route.ts, src/app/api/agents/[agentId]/route.ts

Recommendation: Keep owner/built-in checks on both reads and relationship writes.

Acceptance test: Attempt direct reads and attachment with another user's skill identifier and prove denial without existence leakage.

F-010 - New or empty agents inherited every tool

Severity	Status	Confidence	Category
high	fixed_in_v8	high	capability-security

Finding: Agent creation and runtime fallback could translate absent or empty tool configuration into the full registry.

Impact: A minimally configured agent could unexpectedly gain network or write authority.

Evidence: src/lib/engine/tools/index.ts, src/app/api/agents/route.ts, src/lib/engine/start.ts

Recommendation: Keep an explicit low-risk default allowlist and treat an empty list as no capabilities.

Acceptance test: Create agents with omitted and empty tool lists; prove only documented tools can execute.

F-011 - Catch-all policy allowed medium-risk actions automatically

Severity	Status	Confidence	Category
high	fixed_in_v8	high	human-oversight

Finding: The default policy set included a broad catch-all automatic path that weakened the documented fallback behavior.

Impact: Risky actions could bypass human review despite the UI implying otherwise.

Evidence: src/lib/engine/policy.ts, tests/policy.test.ts

Recommendation: Keep fail-closed fallback policy and require explicit, narrow auto-approval rules.

Acceptance test: Unknown and medium/high-risk actions resolve to human review or denial in policy tests.

F-012 - Runs and approvals have no complete wall-clock deadline model

Severity	Status	Confidence	Category

high	partially_mitigated	high	reliability

Finding: Runs now accept explicit wall-clock deadlines and output/context budgets; approvals persist an expiry timestamp and are expired by local timers and boot recovery. A durable timer service and restart-safe continuation are still absent.

Impact: A run can occupy resources indefinitely or remain pending forever after user abandonment.

Evidence: src/lib/engine/runner.ts, src/app/api/approvals/[approvalId]/route.ts, src/lib/db/schema.ts, src/app/api/runs/route.ts, src/lib/db/index.ts, drizzle/0001_v10_hardening.sql

Recommendation: Add persisted deadlines, timeout ownership, cancellation propagation, and deterministic terminal states.

Acceptance test: Advance a fake clock across model, tool, approval, and run deadlines and prove exact terminal events and released leases.

F-013 - Run event sequence allocation is race-prone

Severity	Status	Confidence	Category
high	partially_mitigated	high	integrity

Finding: Run event sequence allocation and insertion now execute under BEGIN IMMEDIATE with a unique run/sequence constraint and an atomic outbox record. Native multi-process contention tests remain blocked in this environment.

Impact: Concurrent writers can duplicate or reorder sequence numbers and break replay evidence.

Evidence: src/lib/engine/events.ts, src/lib/db/schema.ts, drizzle/0001_v10_hardening.sql, scripts/validate-migrations.mjs

Recommendation: Use a transactionally allocated sequence or append-only event table with a unique run/sequence constraint and retry.

Acceptance test: Run concurrent appenders and prove contiguous unique ordering under load and injected transaction retries.

F-014 - Audit hash-chain append is race-prone and not externally anchored

Severity	Status	Confidence	Category
high	partially_mitigated	high	audit-integrity

Finding: Audit hash-chain read, hash calculation, append, and outbox insertion now run in one immediate transaction. External anchoring and native concurrent-writer tests remain.

Impact: Concurrent events can fork the chain; a database administrator can rewrite the entire chain and recompute hashes.

Evidence: src/lib/engine/audit.ts, scripts/verify-audit.ts, src/lib/db/schema.ts, drizzle/0001_v10_hardening.sql

Recommendation: Serialize append, add uniqueness, sign periodic roots, and optionally export roots to independent storage.

Acceptance test: Generate concurrent audit writes, verify one linear chain, then mutate history and prove signed-root verification fails.

F-015 - Disabled provider configurations were still eligible

Severity	Status	Confidence	Category
high	fixed_in_v8	high	provider-routing

Finding: Provider activation status was not consistently enforced during dispatch or provider tests.

Impact: An operator could believe a provider was disabled while the system still sent data to it.

Evidence: src/lib/engine/gateway.ts, src/lib/engine/runner.ts, src/app/api/providers/test/route.ts

Recommendation: Retain enabled checks at every provider resolution boundary.

Acceptance test: Disable each provider and prove dispatch, health test, and fallback routing never contact it.

F-016 - Database migration failure was tolerated

Severity	Status	Confidence	Category
high	fixed_in_v8	high	startup-integrity

Finding: Migration errors were logged but startup could continue against an unknown schema.

Impact: The application could write incompatible data or fail later in less diagnosable ways.

Evidence: src/lib/db/index.ts

Recommendation: Keep startup fail-closed for required schema migrations.

Acceptance test: Force a migration failure and prove the process refuses readiness without serving mutation routes.

F-017 - Fetched content remains a prompt-injection channel

Severity	Status	Confidence	Category
high	partially_mitigated	high	agent-security

Finding: Fetched data is now labeled untrusted and egress-limited, but untrusted text still enters model context without a fully enforced authority separation or content transformation policy.

Impact: A malicious page can attempt to override instructions, induce tool use, or exfiltrate context.

Evidence: src/lib/engine/tools/httpFetch.ts, src/lib/engine/runner.ts, docs/canonical/07_SECURITY_TRUST_SAFETY.md

Recommendation: Separate data from instructions, apply trust labels in the internal protocol, and require policy authorization independent of model text.

Acceptance test: Use adversarial fetched pages that request secrets and destructive tools; prove zero authority escalation and no protected-context leakage.

F-018 - Native database integration tests cannot run in the constrained environment

Severity	Status	Confidence	Category
high	environment_blocked	high	verification

Finding: The safe install deliberately skips native post-install scripts, leaving better-sqlite3 unavailable for three database-backed suites.

Impact: Critical approval expiry, audit, and runner integration behavior is not independently executed in this environment.

Evidence: docs/roast/evidence/v8-full-tests.txt, docs/canonical/24_HONEST_LIMITATIONS.md

Recommendation: Run the blocked suites on an approved clean Linux CI image with pinned Node headers and a trusted prebuilt/native toolchain.

Acceptance test: All database suites pass from a clean lockfile install and publish logs, environment digest, and database reset evidence.

F-019 - Production build did not complete end-to-end in the audit environment

Severity	Status	Confidence	Category
high	environment_blocked	high	release-verification

Finding: The production build compiles and completes type validation, but page-data collection cannot load the absent better-sqlite3 native binding in the constrained environment.

Impact: A source-level and compile-level green result does not prove deployable application startup or route data collection.

Evidence: docs/canonical/evidence/v10-build.txt, scripts/build-production.mjs

Recommendation: Run the build with the verified native binding on a trusted CI or development host, then execute startup and browser E2E gates.

Acceptance test: A clean install completes the production build, starts the server, and passes Playwright without hidden network or native-binding errors.

F-020 - User data export, correction, and deletion are not implemented end-to-end

Severity	Status	Confidence	Category
high	partially_mitigated	high	data-governance

Finding: User export, password-confirmed deletion, owned-data cascade, governed-memory correction/deletion, and logical audit redaction overlays are implemented. Native end-to-end deletion, backup propagation, and legal retention validation remain.

Impact: Users cannot exercise data rights reliably and deleted data may survive in logs, artifacts, or backups.

Evidence: docs/canonical/20_DATA_LIFECYCLE_CONTRACT.md, src/lib/db/schema.ts, src/app/api/account/export/route.ts, src/app/api/account/delete/route.ts, src/app/api/memory/route.ts, src/lib/engine/audit.ts, docs/canonical/data/DATA_INVENTORY.json

Recommendation: Implement data inventory-backed export and deletion workflows with tombstone/backup semantics and audit evidence.

Acceptance test: Create a user with all data classes, export it, delete it, search every primary/derived store, restore an older backup, and prove policy-compliant handling.

F-021 - Custom templates are globally visible and mutable

Severity	Status	Confidence	Category
high	fixed_in_v10	high	identity-tenancy

Finding: Custom templates now carry an owner identifier; listing and creation expose only built-in templates plus the authenticated user's templates.

Impact: Private workflows or proprietary prompts can be exposed or changed across users.

Evidence: src/lib/db/schema.ts, src/app/api/templates/route.ts, drizzle/0001_v10_hardening.sql

Recommendation: Separate immutable built-ins from tenant-owned custom templates and scope every query.

Acceptance test: Two-user route tests prove private templates cannot be listed, read, updated, or referenced cross-tenant.

F-022 - Audit access is restricted but audit data remains globally modeled

Severity	Status	Confidence	Category
high	partially_mitigated	high	identity-tenancy

Finding: The route now restricts entries to administrators, but audit rows lack a complete tenant/workspace ownership model and delegated audit roles.

Impact: A global administrator becomes a high-value data aggregation point and tenant-level audit cannot be safely delegated.

Evidence: src/app/api/audit/verify/route.ts, src/lib/db/schema.ts

Recommendation: Add tenant/workspace dimensions and explicit audit-reader roles while preserving system-level security audit separation.

Acceptance test: Tenant auditors see only their workspace; system auditors see redacted cross-tenant metadata under separate authorization.

F-023 - Registration remains open with weak abuse controls

Severity	Status	Confidence	Category

high	fixed_in_v10	high	authentication

Finding: Registration now bootstraps only the first administrator by default, remains closed afterward unless explicitly enabled, supports an invite token, enforces a twelve-character minimum, and uses rate limiting without trusting proxy headers by default.

Impact: An exposed installation can be populated by unauthorized users and amplified against local or paid model resources.

Evidence: src/app/api/register/route.ts, src/lib/auth/password.ts, src/lib/rate-limit.ts, .env.example

Recommendation: Add explicit registration mode, invitation/first-owner bootstrap, stronger password checks, throttling, and non-enumerating responses.

Acceptance test: Unauthenticated registration is denied in closed mode; invite reuse, enumeration, weak passwords, and burst attempts fail safely.

F-024 - Secret encryption lacks documented rotation and context binding

Severity	Status	Confidence	Category
high	partially_mitigated	medium	cryptography

Finding: Provider secrets now use versioned AES-GCM records, HKDF purpose separation, and user/provider context binding with legacy decryption compatibility. Automated rotation, revocation, and custody recovery are not complete.

Impact: A leaked master key or ciphertext relocation can expose all provider credentials with limited recovery options.

Evidence: src/lib/crypto.ts, docs/canonical/07_SECURITY_TRUST_SAFETY.md, src/app/api/providers/route.ts, src/lib/engine/runner.ts, tests/crypto.test.ts

Recommendation: Introduce key IDs, rotation, AEAD associated data, scoped decrypt authority, secure backup handling, and revocation drills.

Acceptance test: Rotate from key A to B without downtime, prove old ciphertext migration, reject swapped record context, and restore credentials securely.

F-025 - Run creation has no client idempotency contract

Severity	Status	Confidence	Category
high	partially_mitigated	high	api-reliability

Finding: Run creation accepts a user-scoped Idempotency-Key, serializes create-or-reuse under BEGIN IMMEDIATE, rejects key reuse with a different request, and has a unique database constraint. Native concurrent API tests remain blocked.

Impact: Network retries or double clicks can duplicate expensive or destructive work.

Evidence: src/app/api/runs/route.ts, src/lib/db/schema.ts, src/lib/engine/runner.ts, drizzle/0001_v10_hardening.sql

Recommendation: Accept scoped idempotency keys, persist request/result hashes, and return the original run for safe retries.

Acceptance test: Send the same request concurrently and repeatedly; prove exactly one run and a deterministic replay response.

F-026 - Approval expiry is not owned by a durable timer

Severity	Status	Confidence	Category
high	partially_mitigated	high	human-oversight

Finding: Approval expiry is persisted and enforced by local wait timers and startup sweeping, but no independent durable timer worker owns expiry while the application is offline.

Impact: Expired approvals can remain pending or resume ambiguously, blocking work and weakening policy guarantees.

Evidence: src/lib/engine/runner.ts, src/app/api/approvals/[approvalId]/route.ts, tests/approval-expiry.test.ts, src/lib/db/index.ts, src/lib/db/schema.ts

Recommendation: Persist approval deadlines and let a durable worker own expiry, cancellation, and notification.

Acceptance test: Restart before and after expiry and prove one deterministic expired transition and no later execution.

F-027 - Restart semantics for pending approvals are incomplete

Severity	Status	Confidence	Category
high	partially_mitigated	high	recovery

Finding: A serializable revision-checked approval wait contract now proves one-decision and expiry semantics, but the active runner still cannot resume it after restart.

Impact: An approved action may never resume or may be reissued incorrectly after restart.

Evidence: src/lib/engine/runner.ts, src/lib/db/schema.ts, src/lib/durable/approval-continuation.ts, tests/approval-continuation.test.ts

Recommendation: Persist the wait contract, durable timer, continuation pointer, and requeue behavior in the active runner.

Acceptance test: Restart with a pending approval, decide it from another process, and prove the exact continuation resumes once.

F-028 - Configured maximum plan size was not enforced

Severity	Status	Confidence	Category
high	fixed_in_v8	high	resource-governance

Finding: The planner schema previously accepted plans beyond the configured step limit.

Impact: A model could create runaway work graphs and consume disproportionate resources.

Evidence: src/lib/engine/plan.ts, tests/plan.test.ts

Recommendation: Keep max-step enforcement at parse/validation time.

Acceptance test: Generate plans at limit, limit+1, and extreme sizes; prove bounded parse time and clear rejection.

F-029 - Tool iteration exhaustion could be treated as success

Severity	Status	Confidence	Category
high	fixed_in_v8	high	runtime-correctness

Finding: A step could leave the model still requesting tools after the loop budget but continue toward completion.

Impact: Incomplete or looping work could be presented as successful.

Evidence: src/lib/engine/runner.ts

Recommendation: Fail the step and run with explicit budget-exhausted evidence.

Acceptance test: Use a model fixture that always requests another tool and prove deterministic failure at the configured limit.

F-030 - Usage accounting could double-count tokens

Severity	Status	Confidence	Category
medium	fixed_in_v8	high	economics

Finding: Token usage could be recorded both through provider events and response usage aggregation.

Impact: Cost, capacity, and model-routing decisions could be based on inflated telemetry.

Evidence: src/lib/engine/runner.ts, src/lib/engine/event-types.ts

Recommendation: Use one authoritative accounting path with reconciliation fields.

Acceptance test: Replay a response with provider usage events and summary usage; prove totals equal the provider bill once.

F-031 - Rejected approval did not close the tool evidence chain

Severity	Status	Confidence	Category
medium	fixed_in_v8	high	evidence-integrity

Finding: A rejected action produced a model-facing result but no matching tool.result event.

Impact: Evidence reconstruction reported an unmatched call and could not explain the human decision path.

Evidence: src/lib/engine/runner.ts, src/lib/engine/evidence-summary.ts

Recommendation: Emit a terminal tool result representing rejection without executing the tool.

Acceptance test: Reject an approval and prove call/result pairing, no side effect, and visible human-decision evidence.

F-032 - Prompts and objectives are duplicated into persistent evidence without minimization

Severity	Status	Confidence	Category
medium	partially_mitigated	high	privacy

Finding: Full system prompts and run objectives are no longer duplicated into run-start events or audit payloads; hashes and character counts are stored instead. The objective remains once in the run record because it is operational input, and broader retention controls remain.

Impact: Sensitive business data can persist longer and in more stores than users expect.

Evidence: src/lib/engine/runner.ts, src/lib/engine/audit.ts, src/lib/engine/events.ts, src/app/api/account/export/route.ts

Recommendation: Store references/hashes and redacted summaries by default; define an explicit secure evidence retention mode for full content.

Acceptance test: Insert known sensitive markers and prove default logs/events/audit omit them while authorized artifact storage remains traceable.

F-033 - Artifact writes had no practical size ceiling

Severity	Status	Confidence	Category
medium	fixed_in_v8	high	resource-governance

Finding: The write-artifact tool accepted unbounded content.

Impact: A model could exhaust memory, disk, database capacity, or UI rendering.

Evidence: src/lib/engine/tools/writeArtifact.ts

Recommendation: Keep a documented byte cap and later add streaming/object storage where justified.

Acceptance test: Attempt boundary, boundary+1, and multibyte payloads and prove exact size enforcement.

F-034 - Policy regular expressions can create denial of service

Severity	Status	Confidence	Category
medium	fixed_in_v10	medium	policy-engine

Finding: Policy patterns pass a bounded safe-regex compiler that rejects oversized expressions, backreferences, lookarounds, nested quantifiers, repeated wildcards, and malformed patterns before evaluation.

Impact: Catastrophic backtracking can block the Node.js event loop during authorization.

Evidence: src/lib/engine/policy.ts, src/app/api/policies/route.ts, src/lib/security/safe-regex.ts, tests/safe-regex.test.ts

Recommendation: Use constrained glob/pattern semantics or safe-regex validation and bounded evaluation.

Acceptance test: Submit known pathological expressions and long inputs; prove bounded latency or rejection.

F-035 - Rate limiting is process-local and trusts client forwarding headers

Severity	Status	Confidence	Category
medium	partially_mitigated	high	abuse-control

Finding: Forwarding headers are ignored unless proxy trust is explicitly enabled. Rate-limit state remains process-local and therefore is not a distributed production control.

Impact: Attackers can bypass limits across workers/restarts or by rotating forged header values.

Evidence: src/lib/rate-limit.ts, src/app/api/runs/route.ts, src/app/api/providers/test/route.ts, tests/rate-limit.test.ts

Recommendation: Use authenticated principal/workspace keys and a shared limiter or documented single-process boundary; configure trusted proxies explicitly.

Acceptance test: Exercise multiple workers, restarts, spoofed headers, users, and IPs; prove the intended global and per-user budget.

F-036 - Security headers improved but CSP remains absent

Severity	Status	Confidence	Category
medium	partially_mitigated	high	browser-security

Finding: The application now sends a Content Security Policy and baseline headers. The compatibility policy still permits inline scripts and styles, and browser-level enforcement tests remain.

Impact: A future injection bug has fewer browser-side containment barriers.

Evidence: next.config.ts

Recommendation: Add nonce/hash-based CSP, report-only rollout, and route tests without breaking Next.js runtime behavior.

Acceptance test: Production responses include the approved CSP and Playwright confirms no required script/style violations.

F-037 - Source lint is intentionally shallow and build ignores ESLint

Severity	Status	Confidence	Category
medium	partially_mitigated	high	code-quality

Finding: Build no longer suppresses ESLint and the non-interactive source lint checks merge markers, dangerous directives, dynamic code execution, malformed JSON, and client/server boundaries. A full semantic ESLint ruleset remains absent.

Impact: Common correctness, React, security, and accessibility defects can pass CI.

Evidence: scripts/lint-source.mjs, next.config.ts, package.json

Recommendation: Adopt a pinned non-interactive ESLint configuration plus targeted security/accessibility rules and keep the fast scanner as an additional gate.

Acceptance test: A clean install runs lint non-interactively; seeded violations fail with stable diagnostics.

F-038 - No generated SBOM, model inventory, or build provenance

Severity	Status	Confidence	Category
high	fixed_in_v10	high	supply-chain

Finding: The release pipeline generates a CycloneDX SBOM, license inventory, model inventory, and source-tree build provenance with hashes.

Impact: Operators cannot reliably identify vulnerable components, licenses, models, plugins, or reproduce the artifact.

Evidence: package.json, docs/canonical/14_ACCEPTANCE_GATES.md, scripts/generate-release-evidence.mjs, docs/canonical/evidence/release/SBOM.cyclonedx.json, docs/canonical/evidence/release/LICENSE_INVENTORY.json, docs/canonical/evidence/release/MODEL_INVENTORY.json, docs/canonical/evidence/release/BUILD_PROVENANCE.json

Recommendation: Generate CycloneDX/SPDX, license inventory, model/plugin hashes, and build provenance during release.

Acceptance test: The extracted ZIP contains validated inventories matching the lockfile and shipped artifacts, signed or hashed in the release manifest.

F-039 - Observability lacks traces, SLOs, and capacity evidence

Severity	Status	Confidence	Category
high	partially_mitigated	high	observability

Finding: A local metrics endpoint now reports status counts, latency percentiles, token usage, approval expiry, outbox backlog, durable-job state, and SLO evaluations. Cross-boundary traces, energy measurement, historical capacity series, and telemetry retention remain.

Impact: Operators cannot answer why work is slow, expensive, failing, or saturating resources.

Evidence: src/lib/engine/event-types.ts, docs/canonical/09_EVALUATION_OBSERVABILITY.md, src/lib/observability/metrics.ts, src/app/api/metrics/route.ts, tests/metrics.test.ts

Recommendation: Add redacted trace correlation, metrics, usage reconciliation, queue measures, SLOs, and local dashboards.

Acceptance test: A benchmark run yields traceable p50/p95/p99 latency, queue depth, failure classes, token/cost totals, and an SLO verdict.

F-040 - Governed memory architecture is documentation-only

Severity	Status	Confidence	Category
high	fixed_in_v11	high	memory

Finding: Governed memory is implemented and integrated into prompt assembly with owner filtering, expiry, token budgets, quarantine, provenance labeling, corrections, and deletion.

Impact: Long-running agents either forget important context or accumulate untrusted, stale, contradictory, or cross-user material.

Evidence: docs/canonical/06_DATA_MEMORY_KNOWLEDGE.md, src/lib/db/schema.ts, src/lib/memory/governed-memory.ts, src/app/api/memory/route.ts, src/components/memory/memory-ledger.tsx, tests/governed-memory.test.ts, docs/canonical/decisions/ADR-0022-governed-local-memory-baseline.md, src/lib/memory/prompt-context.ts, tests/memory-prompt-context.test.ts

Recommendation: Maintain ablation and poisoning evaluations before expanding automatic memory creation.

Acceptance test: Create contradictory, expired, private, corrected, and deleted memories; prove scoring, provenance, isolation, and removal behavior.

F-041 - Provider selection is not a measured capability router

Severity	Status	Confidence	Category
high	fixed_in_v11	high	model-routing

Finding: Model capability profiles, hard eligibility constraints, deterministic utility scoring, explicit fallback, no-eligible failure, persistence, API control, and run-level rationale are implemented.

Impact: Critical work can be dispatched by static configuration rather than demonstrated fitness.

Evidence: src/lib/engine/gateway.ts, src/lib/engine/providers, docs/canonical/05_REASONING_AND_INTELLIGENCE.md, src/lib/models/router.ts, src/lib/models/db-routing.ts, tests/model-router.test.ts

Recommendation: Populate profiles through repeatable benchmark runs and invalidate stale evaluations.

Acceptance test: Given constrained tasks and unavailable models, route to the Pareto-eligible candidate or return no-eligible-model without silent downgrade.

F-042 - Test portfolio is too narrow for the threat and failure model

Severity	Status	Confidence	Category
high	partially_mitigated	high	evaluation

Finding: The safe portfolio now includes lifecycle, evidence, egress, durable execution, backup/restore, memory, regex, tool-schema, metrics, context-budget, graph, crypto, and policy tests. Native database, browser, accessibility, load, and broad fault-injection suites remain incomplete.

Impact: The system can pass 47 safe tests while remaining unsafe under realistic operation.

Evidence: tests, e2e, docs/canonical/14_ACCEPTANCE_GATES.md, tests/application-backup.test.ts, tests/governed-memory.test.ts, tests/metrics.test.ts, tests/context-budget.test.ts

Recommendation: Build layered adversarial and fault-injection suites tied to every high/critical finding.

Acceptance test: Each high/critical finding has a failing regression before repair and a passing independent verification after repair.

F-043 - End-to-end tests are brittle smoke checks

Severity	Status	Confidence	Category
medium	partially_mitigated	high	evaluation

Finding: The Playwright suite now uses operator-supplied credentials, semantic readiness conditions, and coverage for run evidence, agent evaluation, and governed-memory lifecycle. Isolated database fixtures, repeated parallel/serial execution, accessibility assertions, and measured flake-rate evidence remain.

Impact: The suite is materially less brittle, but shared-state and unexecuted browser coverage can still conceal authorization, recovery, accessibility, or concurrency regressions.

Evidence: e2e/smoke.spec.ts, playwright.config.ts, .github/workflows/ci.yml

Recommendation: Add per-run isolated databases and users, deterministic fixture teardown, accessibility checks, authorization negatives, restart recovery, and repeated serial/parallel flake measurement.

Acceptance test: Run the E2E suite repeatedly in serial and parallel modes against isolated state, including accessibility, authorization, offline, and recovery cases, with a bounded measured flake rate.

F-044 - Seed workflow is destructive and includes known demo credentials

Severity	Status	Confidence	Category
medium	fixed_in_v10	high	operations

Finding: The seed refuses to run unless a destructive-seed flag is set, requires an operator-supplied twelve-character demo password, and no longer contains fallback application secrets or prints credentials.

Impact: Running it in the wrong environment can destroy data or create an easily guessed account.

Evidence: scripts/seed.ts, playwright.config.ts, .env.example

Recommendation: Require explicit development/test environment, random ephemeral credentials, targeted fixture cleanup, and destructive confirmation guards.

Acceptance test: Attempt seed in production mode and against a non-empty protected database; prove hard refusal and no mutation.

F-045 - Evidence percentage measures structure, not truth

Severity	Status	Confidence	Category
medium	partially_mitigated	high	evidence-quality

Finding: The UI originally risked presenting event-shape completeness as overall evidence quality. It is now labeled structural coverage, but semantic correctness and verifier independence remain separate unfinished controls.

Impact: Users can over-trust a complete-looking ledger even when outputs are wrong or self-verified.

Evidence: src/components/runs/evidence-ledger.tsx, src/lib/engine/evidence-summary.ts

Recommendation: Keep structural labeling and add claim-level evidence, verifier identity, reproducibility, contradiction, and semantic evaluation.

Acceptance test: Show a structurally complete but deliberately false run and prove the UI does not label it verified.

F-046 - Most canonical capabilities remain blocked or unimplemented

Severity	Status	Confidence	Category
high	partially_mitigated	high	program-execution

Finding: v9 and v10 implement durable contracts, SQLite persistence primitives, transactional outbox, governed memory, encrypted restore, lifecycle controls, observability, data export/deletion, budgets, and release provenance. Production worker cutover, measured routing, delegated identity, and full native/browser verification remain.

Impact: Documentation volume can be mistaken for platform maturity, obscuring the implementation gap.

Evidence: docs/canonical/tasks/TASKS.json, docs/canonical/16_TRACEABILITY_MATRIX.md, docs/canonical/26_V9_DURABLE_ENGINEERING_REPORT.md, src/lib/durable/sqlite-store.ts, src/lib/backup/application-backup.ts, src/lib/memory/governed-memory.ts

Recommendation: Expose implemented-versus-designed status prominently and execute dependency-unblocking vertical slices rather than adding parallel documents.

Acceptance test: A release dashboard derives status from code/test evidence and cannot count designed-only tasks as delivered capability.

F-047 - SQLite and web-process topology limit safe scale and failover

Severity	Status	Confidence	Category
high	partially_mitigated	high	infrastructure

Finding: SQLite now has immediate-transaction serialization, persisted durable schemas, a repository-owned adapter, outbox leases, and migration checks. The web process still launches the production runner and there is no tested worker fleet or failover topology.

Impact: Multi-process deployment, failover, concurrent writers, and horizontal scaling can corrupt assumptions or lose work.

Evidence: src/lib/db/index.ts, src/lib/engine/start.ts, docs/canonical/08_INFRASTRUCTURE_SOVEREIGNTY.md, src/lib/durable/sqlite-store.ts, src/lib/durable/outbox.ts, src/lib/engine/runner.ts, scripts/validate-migrations.mjs

Recommendation: Define supported single-node limits, separate durable worker execution, and introduce a tested persistence profile before scaling out.

Acceptance test: Run load and failover tests at documented limits; unsupported topologies must fail configuration validation.

F-048 - No tested Content Security Policy

Severity	Status	Confidence	Category
high	partially_mitigated	high	browser-security

Finding: A CSP is configured and scanned, but it retains unsafe-inline compatibility directives and has not passed browser-level nonce, third-party, or violation-report testing.

Impact: Cross-site scripting impact is not meaningfully contained at the browser policy layer.

Evidence: next.config.ts, scripts/roast-project.mjs

Recommendation: Design and roll out report-only then enforced CSP with route-level automated checks.

Acceptance test: All production pages load under enforced CSP with no unauthorized script, style, frame, connect, image, or font source.

F-049 - Tool arguments are not uniformly governed by explicit versioned schemas

Severity	Status	Confidence	Category
high	fixed_in_v11	high	capability-security

Finding: Tools declare versioned schemas and capabilities; arguments are validated, allowlists enforced, and execution is bounded by timeout and output size.

Impact: Malformed, oversized, ambiguous, or injected arguments can reach tool implementations inconsistently.

Evidence: src/lib/engine/tools/index.ts, src/lib/engine/types.ts, src/lib/engine/tools/validate.ts, src/lib/engine/runner.ts, tests/tool-arguments.test.ts, src/lib/engine/tool-runtime.ts, tests/tool-runtime.test.ts

Recommendation: Add OS-level isolation for high-risk tools and maintain mutation tests for schema bypass.

Acceptance test: Fuzz every tool with invalid types, extra fields, oversized values, Unicode edge cases, and schema-version mismatch.

F-050 - Context and output budgets are incomplete

Severity	Status	Confidence	Category
high	partially_mitigated	high	context-engineering

Finding: Run requests now bound context and output tokens; context assembly deterministically preserves mandatory messages and recent complete segments under budget. Adaptive stopping, cache isolation, and omission-quality benchmarks remain.

Impact: Long histories or retrieved content can crowd out policy/current facts, increase cost, and degrade correctness.

Evidence: src/lib/engine/runner.ts, docs/canonical/05_REASONING_AND_INTELLIGENCE.md, src/app/api/runs/route.ts, src/lib/engine/context-budget.ts, tests/context-budget.test.ts

Recommendation: Implement mandatory-context reservation, measured selection, output ceilings, and bounded planner/critic/verifier stopping.

Acceptance test: Run adversarial long-context cases and prove mandatory policy/current code facts survive while cost and latency remain within budget.

F-051 - Stale workers could commit after lease takeover when identity was reused

Severity	Status	Confidence	Category
critical	fixed_in_v12	high	recovery failure

Finding: Durable mutations previously checked worker identity and lease time but did not bind commits to a monotonically increasing fencing token. A restarted worker using the same identity could race a newer lease holder.

Impact: Split-brain workers could overwrite checkpoints or complete a job after losing authority, invalidating exactly-once and recovery claims.

Evidence: src/lib/durable/types.ts, src/lib/durable/memory-store.ts, src/lib/durable/sqlite-store.ts, tests/durable-fencing.test.ts, tests/v12-property-race.test.ts

Recommendation: Require a fencing token on every lease-bound mutation and reject stale tokens in memory and SQLite adapters.

Acceptance test: Acquire a lease twice with the same worker identity after expiry; token 1 must be rejected and token 2 must commit.

F-052 - Approval resolution lacked a cross-process compare-and-set revision

Severity	Status	Confidence	Category
high	fixed_in_v12	high	agent-control failure

Finding: The active approval row had a status but no revision. Competing processes could both observe pending before one committed a terminal decision.

Impact: A stale operator action or concurrent process could overwrite or contradict the authoritative human decision.

Evidence: src/lib/db/schema.ts, src/lib/engine/runner.ts, src/app/api/approvals/[approvalId]/route.ts, drizzle/0006_v12_fencing_and_verification.sql

Recommendation: Persist an approval revision and update terminal status only when id, prior status, and revision all match. Return a conflict to stale clients.

Acceptance test: Two resolutions using the same expected revision must yield one successful transition and one approval_revision_conflict.

F-053 - Verification decisions were not immutably bound to tested source and release artifacts

Severity	Status	Confidence	Category
high	fixed_in_v12	high	verification failure

Finding: Run evidence and human review existed, but there was no immutable attestation binding requirement revision, source tree, test output, artifact, builder, verifier, and release candidate.

Impact: A verification label could outlive or refer to different code and test evidence, creating false release confidence.

Evidence: src/lib/verification/attestation.ts, src/app/api/runs/[runId]/verify/route.ts, src/components/runs/verification-panel.tsx, tests/verification-attestation.test.ts

Recommendation: Create hash-bound verification attestations, reject builder/verifier identity collisions, and expose absent verification in the interface.

Acceptance test: Changing any bound field must invalidate the evidence hash; self-verification must be rejected.

F-054 - Runtime calm could be mistaken for release or pilot readiness

Severity	Status	Confidence	Category
high	fixed_in_v12	high	design deception

Finding: Operational metrics showed local state but did not explicitly gate build, native tests, browser E2E, accessibility, restart recovery, tenant isolation, or extracted-package evidence.

Impact: An operator could interpret zero failures in an empty or lightly used system as evidence that the release was pilot-ready.

Evidence: src/lib/release/pilot-readiness.ts, src/components/operations/control-room.tsx, tests/pilot-readiness.test.ts

Recommendation: Add a conservative evidence gate that treats unknown release evidence as blocking and separates runtime health from release classification.

Acceptance test: Healthy empty metrics with unknown release facts must remain DEVELOPMENT ONLY; controlled pilot requires every hard fact.

F-055 - Model routing did not distinguish verification quality or unhealthy candidates

Severity	Status	Confidence	Category
high	fixed_in_v12	high	agent-control failure

Finding: The measured router scored planning and coding capabilities but lacked an explicit verifier-quality signal and health state.

Impact: Verification tasks could select a strong builder but weak reviewer, or choose an unavailable model if other metrics were attractive.

Evidence: src/lib/models/router.ts, src/lib/models/db-routing.ts, src/app/api/models/route.ts, tests/model-router.test.ts

Recommendation: Record verification quality and health, apply task-class weighting, and reject unavailable candidates as a hard constraint.

Acceptance test: A verification task must select the eligible high-verification model and reject an unavailable higher-scoring model.

16. Supply Chain and Release Integrity

- CycloneDX SBOM, license inventory, model inventory, source provenance, release manifest, and SHA-256 are generated from the final tree.
- Dependencies, build output, databases, credentials, caches, and local artifacts are excluded from the source ZIP.
- The Drizzle version is not changed merely to make the Roast green; acquisition and full validation of the real candidate package remain required.

17. Test and Gate Evidence

Gate	Status	Detail
canonical	**passed**	65 tasks, 68 claims, 28 sources, 26 ADRs, 28 domains, 15 experiments
english_only	**passed**	
source_lint	**passed**	
migration_validation	**passed**	7 migrations, 12 required tables
typescript	**passed**	
safe_tests	**passed**	40 files, 140 tests
baseline_evaluation	**passed**	8/8
durable_crash_proof	**passed**	2 worker attempts, exactly 1 side effect
static_roast	**partial**	43/44; Drizzle upgrade remains open
dynamic_roast	**passed**	6/6
targeted_mutation	**passed**	6/6 curated semantic mutants killed
native_sqlite_tests	**environment_blocked**	better_sqlite3.node absent; 3 suites fail before test collection
production_build	**environment_blocked**	compiled and typechecked; page-data collection exceeded bounded window
browser_screenshot_e2e	**environment_blocked**	Chromium timed out due container kernel/DBus constraints
browser_accessibility	**not_verified**	
independent_verifier	**not_verified**	

18. Honest Limitations and Release Decision

- Live-provider step continuation is not restart-safe.
- Approval persistence does not reconstruct provider conversation state.
- Full tenant/workspace isolation is incomplete.
- Three native suites cannot collect without better_sqlite3.node.
- Production build did not complete page-data collection in the bounded window.
- Browser E2E, WCAG, and visual regression did not run.
- No independent verifier signed the final artifact.

Final classification: **DEVELOPMENT ONLY**

19. Next Canonical Work

- Prove live-provider step continuation and reconciliation under restart.
- Run native multi-process SQLite lease, approval, audit, and runner tests in transparent CI.
- Complete tenant/workspace authorization and mutation coverage.

- Acquire and validate a secure Drizzle upgrade with rollback.
- Complete production build, browser E2E, WCAG accessibility, and visual regression.
- Bind a separately operated verifier attestation to the final release hash.